A Hybrid Post-Quantum KEM: RSA-2048 (Furui-Sieved Primes) + ML-KEM-768

Construction, Security, and an Implementer's Guide

Ryoji Furui — Ryoji.info

June 2026

Abstract

This paper describes a *hybrid* key-encapsulation mechanism (KEM): a way for two parties to agree on a shared 32-byte secret that stays safe as long as **either** of two very different building blocks remains unbroken. The first building block is classical **RSA-2048**. The second is **ML-KEM-768**, the lattice-based KEM that NIST standardized as FIPS 203. We run both, then mix their two secrets with HKDF-SHA-256 to produce one session key. The pay-off is simple: an attacker has to break *both* RSA and ML-KEM to learn the session key, so the scheme protects you today (when RSA is strong) and continues to protect you after a quantum computer breaks RSA (when ML-KEM carries the load).

A second, independent idea appears here only as an engineering optimization: the **Furui 6n±1 sieve**, a fast way to generate RSA primes. We are explicit that the sieve contributes *nothing* to security — it just makes classical key generation cheaper during the migration period, when systems keep minting new RSA keys. We prove this carefully so that no reader mistakes a performance trick for a security feature.

The paper is written for engineers who want to deploy a hybrid KEM correctly. The main text explains the construction, the threat model, and how to implement it safely. The appendices give the full formal security reduction, the statistical validation of the sieve, and a reproducible benchmark methodology for readers who want the underlying detail.

> **Authorship and review status.** The 6n±1 prime formulation (Section 5 and Appendix B) is the author's original work [1]. The hybrid construction, security analysis, implementation guidance, and appendices were drafted with the assistance of large language models (Anthropic's Claude, with review comments from Microsoft Copilot) at the author's direction. The author does not have formal training in lattice cryptography or protocol security analysis, and the design has not yet had independent expert review. Treat the construction and all security claims as a well-documented starting point for discussion, not as peer-reviewed guidance, and commission an independent cryptographic review before deploying it. Section 4 explains exactly what rests on assumptions and what would need checking.

How to read this paper

If you are here to **ship something**, read Sections 1–3 (what it is and how it works), Section 6 (implementation, including the hardening checklist in §6.3), and Section 8 (operations). If you are here to **judge the security**, read Section 4 and then Appendix A. If you only care about the **prime-generation trick**, read Section 5 and Appendix B. Notation is kept ASCII-friendly throughout: `||` is concatenation, `pk_R/sk_R` are the RSA public/secret key, `pk_M/sk_M` are the ML-KEM keys, and `ss_RSA/ss_MLKEM` are the two 32-byte component secrets that get combined into the final `ss`.

---

1. Introduction: why a hybrid, and why now

Shor's algorithm [2] lets a large, fault-tolerant quantum computer factor integers and compute discrete logarithms in polynomial time. The moment such a machine exists, the public-key cryptography the internet runs on — RSA, Diffie–Hellman, and the elliptic-curve schemes — stops protecting anything. Public engineering estimates put a *cryptographically relevant quantum computer* (CRQC) — one capable of breaking RSA-2048 — somewhere in the 2030s [3], with wide error bars in both directions.

You do not get to wait for that machine to arrive before acting, for three reasons.

Harvest now, decrypt later. An adversary can record your encrypted traffic *today* and simply keep it until a quantum computer can decrypt it. Any data whose value outlives the arrival of that machine is therefore already at risk. Confidentiality has to be upgraded *before* the threat is real, not after.

Lattice schemes are still young. ML-KEM is excellent and now standardized, but module-lattice cryptography has only been deployable for about a decade. Prudent engineering hedges a young primitive against the possibility of an unforeseen *classical* break.

Regulators already ask for hybrids. Both the U.S. CNSA 2.0 suite [4] and the German BSI's TR-02102 [5] explicitly endorse hybrid modes for the transition period.

A hybrid KEM answers all three at once. It pairs a **classical** component (RSA-2048) with a **post-quantum** component (ML-KEM-768 [6]) and combines them so that the result is secure if *either* one is secure. Before a quantum computer exists, RSA also protects you even if a surprise lattice attack appears. After one exists, ML-KEM protects you even though RSA has fallen. You are never worse off than the stronger of the two components.

What this paper gives you

1. A concrete, implementable construction with full pseudocode for KeyGen, Encaps, Decaps, and the HKDF combiner (Section 3), including exact wire sizes and how to map it onto TLS 1.3 and HPKE.

2. A security argument — intuition in Section 4, full reduction with advantage bounds in Appendix A — showing the hybrid inherits IND-CCA security from whichever component is still standing, plus concrete numbers so you can see the safety margin.
3. A production-oriented implementation guide (Section 6): library choices, side-channel mitigations, and a hardening checklist you can hand to an implementer.
4. Operational guidance (Section 8): a key-rotation policy and a compact playbook for running this through the migration window.
5. The **Furui sieve** for fast RSA prime generation (Section 5), together with a proof and empirical evidence (Appendix B) that its output is statistically indistinguishable from any standard generator — so it is safe to use *and* security-neutral.

One thing to be clear about from the start

The Furui sieve is a **prime-generation primitive, not a cryptographic primitive.** It changes *how fast* you find RSA primes, never *which* primes are reachable or how hard the resulting RSA instance is to break. We include it because efficient classical key generation genuinely matters during a migration in which systems keep generating RSA keys — but it carries no security weight, and Section 4.4 shows it does not even appear in the security proof.

---

2. Background in plain terms

This section gives just enough background to follow the construction. Experts can skip to Section 3.

2.1 What a KEM is

A **key-encapsulation mechanism** is the modern, clean way to do public-key key exchange. It has three algorithms:

- **KeyGen** produces a public key `pk` and a secret key `sk`.
- **Encaps(pk)** takes someone's public key and outputs a ciphertext `ct` together with a fresh shared secret `ss`.
- **Decaps(sk, ct)** takes the matching secret key and the ciphertext and recovers the same `ss`.

The sender runs Encaps and transmits `ct`; the receiver runs Decaps and ends up with the same `ss`. Both sides now hold a shared secret they can feed to an authenticated cipher such as AES-256-GCM. The security goal we care about is **IND-CCA**: even an adversary who can ask for decapsulations of ciphertexts of its choosing cannot distinguish the real `ss` from random.

2.2 RSA-2048 and the quantum threat

RSA-KEM [7] derives a shared secret from a random value encrypted under an RSA public key. Its security rests on the difficulty of factoring the 2048-bit modulus `N = p·q`. Against classical computers that is a safe bet for now. Against a quantum computer running Shor's algorithm it is not a bet at all: the modulus factors in polynomial time, *regardless* of how `p` and `q` were chosen or how large they are. This is why RSA is the *transitional* half of our hybrid — strong today, expected to fall later.

2.3 ML-KEM-768

ML-KEM (FIPS 203 [6]) is the standardized form of the CRYSTALS-Kyber lattice KEM. The "-768" parameter set targets NIST Category 3 (roughly 192-bit) security. Its three algorithms produce a 1184-byte public key, a 2400-byte secret key, a 1088-byte ciphertext, and a 32-byte shared secret. Security rests on the **Module Learning With Errors (M-LWE)** problem, for which no efficient classical *or* quantum attack is known; the best attacks run in exponential time. ML-KEM is the *durable* half of our hybrid — the one expected to survive a quantum computer.

One detail matters for implementers and reappears in Section 3.5: on an invalid ciphertext, ML-KEM does **not** return an error. It returns a deterministic-but-unpredictable value derived from the secret key. This is called **implicit rejection**, and preserving it correctly is essential to security.

2.4 What "hybrid" means

A hybrid KEM runs two component KEMs side by side and combines their shared secrets with a key-derivation function (KDF):

```
ss = KDF( ss_RSA || ss_MLKEM ,  transcript )
```

The combiner is designed so that `ss` looks random to an attacker as long as *at least one* of `ss_RSA`, `ss_MLKEM` looks random to that attacker. This "secure if either survives" property is exactly the insurance we want. It is a standard, well-studied pattern [8], [9], and the same construction the TLS 1.3 hybrid drafts use [10]. Section 4 makes the property precise.

2.5 The Furui sieve, in one paragraph

Every prime greater than 3 has the form 6n−1 or 6n+1. The Furui sieve [1] enumerates only candidates of that form and crosses out the composites among them, which is a compact 2,3-wheel sieve of Eratosthenes. It finds primes faster than a naive scan because it never wastes work on the two-thirds of integers divisible by 2 or 3. That is the entire idea. It produces ordinary primes; it just produces them quickly. Everything security-relevant about RSA is unchanged by using it. We return to it in Section 5 and prove the "unchanged" claim in Appendix B.

---

3. The construction

3.1 Notation

Let `RSA = (RSA.KeyGen, RSA.Encaps, RSA.Decaps)` be RSA-KEM-2048 in the style of RFC 5990 [7], and `MLKEM = (MLKEM.KeyGen, MLKEM.Encaps, MLKEM.Decaps)` be ML-KEM-768. Let `HKDF` be HKDF-SHA-256 [11]. We write `||` for concatenation. Each component produces a 32-byte shared secret; the hybrid combines them into one 32-byte `ss`.

3.2 Key generation

```
HybridKEM.KeyGen():
    (pk_R, sk_R) <- RSA.KeyGen(2048)      # primes via the Furui sieve
(Section 5)
    (pk_M, sk_M) <- MLKEM.KeyGen()
    pk = pk_R || pk_M
    sk = sk_R || sk_M || pk_R || pk_M      # cache public keys for the
transcript
    return (pk, sk)
```

The public key is `256 + 1184 = 1440` bytes (plus length tags). The secret key stores the RSA key, the ML-KEM key, and a cached copy of both public keys for the transcript binding in §3.4 — about 4096 bytes, or 2656 bytes if you recompute the public key from the secret key on demand.

3.3 Encapsulation and decapsulation

```
HybridKEM.Encaps(pk = pk_R || pk_M):
    (ct_R, ss_RSA)   <- RSA.Encaps(pk_R)     # 256-byte ct, 32-byte secret
    (ct_M, ss_MLKEM) <- MLKEM.Encaps(pk_M)   # 1088-byte ct, 32-byte secret
    ct = ct_R || ct_M                        # 1344 bytes
    ss = Combine(ss_RSA, ss_MLKEM, ct_R, ct_M, pk_R, pk_M)
    return (ct, ss)

HybridKEM.Decaps(sk, ct = ct_R || ct_M):
    (sk_R, sk_M, pk_R, pk_M) = parse(sk)
    ss_RSA   <- RSA.Decaps(sk_R, ct_R)
    ss_MLKEM <- MLKEM.Decaps(sk_M, ct_M)     # implicit rejection inside
    ss = Combine(ss_RSA, ss_MLKEM, ct_R, ct_M, pk_R, pk_M)
    return ss
```

3.4 The combiner

The combiner is HKDF-SHA-256 in extract-then-expand form. The **salt** is a fixed label that separates this construction from any other use of HKDF in the same system. The **input key material** is the two component secrets concatenated. The **info** string is the full transcript — both ciphertexts and both public keys — which binds the output to this exact exchange.

```
Combine(ss_RSA, ss_MLKEM, ct_R, ct_M, pk_R, pk_M):
    PRK = HKDF-Extract( salt = "Hybrid-RSA2048-MLKEM768-v1",
                        IKM  = ss_RSA || ss_MLKEM )
```

```
    ss  = HKDF-Expand( PRK,
                       info = ct_R || ct_M || pk_R || pk_M,
                       L    = 32 )
    return ss
```

Four properties make this safe, and each one matters in practice:

1. **Domain separation.** The fixed salt keeps this construction's outputs from ever colliding with another HKDF use in the same deployment.
2. **Transcript binding.** Putting both ciphertexts and both public keys in `info` ties the session key to this specific exchange, which blocks unknown-key-share and re-keying attacks.
3. **"Either survives" security.** Because HMAC-SHA-256 behaves as a PRF keyed by *either* input (the *dual-PRF* property [12]), the output is indistinguishable from random if *either* `ss_RSA` or `ss_MLKEM` is. This is the heart of the hybrid; Section 4 makes it precise.
4. **Versioning.** The `v1` in the salt means a future variant (say RSA-2048 + ML-KEM-1024) produces unrelated outputs, so the two versions can never be confused even by an attacker who controls one component's input. A fixed (rather than per-session) salt is safe here precisely because `info` already carries the per-session transcript.

3.5 Failure handling and why it must be constant-time

This is the part implementers most often get wrong, so it gets its own treatment.

ML-KEM uses implicit rejection: a bad ciphertext yields a deterministic pseudorandom secret derived from `sk_M`, not an error. We make the RSA branch behave the same way: a malformed or out-of-range RSA ciphertext is processed in constant time and yields a pseudorandom 32-byte value derived from `sk_R` and `ct_R`. **Both branches always run, and the combiner always runs**, with whatever each branch returned. The caller cannot tell a fully valid ciphertext from a partly malformed one by looking at the output.

Concretely, decapsulation must satisfy three properties, which together close the chosen-ciphertext side channels:

- **Both branches run in constant time.** Their running time must not depend on whether `ct_R` or `ct_M` is well-formed.
- **The combiner runs unconditionally.** It is called with whatever the two branches returned — a real secret on success, a pseudorandom value on failure — and its 32-byte output is returned with no further checks.
- **No partial-success oracle.** An attacker who submits a good ML-KEM ciphertext with a broken RSA ciphertext (or vice versa) learns only a pseudorandom value bound to the broken input. They never learn *which* component failed, nor anything about the surviving secret.

Hold the failure paths to the same constant-time standard as the success paths. Section 6.2 lists the specific mitigations, and §6.3 turns them into a checklist.

---

4. Why it's secure

This section builds intuition and states the guarantee. The full reduction, with advantage bounds and game hops, is Appendix A.

4.1 The one idea: an "OR" of two problems

Informally: **to break the hybrid, you must break both components.** If RSA-KEM is secure, `ss_RSA` is indistinguishable from random, so — by the dual-PRF property of HKDF — the combined `ss` is indistinguishable from random, no matter what happens to ML-KEM. Symmetrically, if ML-KEM is secure, `ss_MLKEM` carries the day no matter what happens to RSA. The attacker wins only if *both* fail at once.

4.2 Three attackers, and graceful degradation

It helps to picture three kinds of adversary:

- **Today's attacker (classical).** Both RSA and ML-KEM are believed secure, so the hybrid is secure with a comfortable margin (§4.3).
- **A pre-quantum attacker with side quantum gadgets** but no full-scale machine. For our purposes this is the same as the classical case.
- **A post-quantum attacker** running Shor against RSA. RSA's contribution collapses to zero — the attacker recovers `sk_R` and hence `ss_RSA` for any ciphertext. The guarantee then rests entirely on ML-KEM, and the hybrid **degrades gracefully to plain ML-KEM-768 security**. You lose the classical hedge, but you lose nothing else.

4.3 The guarantee, stated plainly

For any efficient adversary `A` against the hybrid's IND-CCA security, there are adversaries `B_R` (against RSA-KEM), `B_M` (against ML-KEM), and `D` (against HKDF's dual-PRF property) of essentially the same running time, such that

```
Adv_hybrid(A)  <=  min( Adv_RSA(B_R),  Adv_MLKEM(B_M) )  +  2 *
Adv_dualPRF(D)
```

Read it as: the hybrid's advantage is at most the **smaller** of the two component advantages (the "either survives" guarantee), plus a small KDF term. Against a quantum attacker `Adv_RSA(B_R)` becomes 1, the `min` selects `Adv_MLKEM(B_M)`, and the bound is dominated by ML-KEM — exactly the graceful degradation of §4.2.

4.4 The Furui sieve is not in the proof

Notice that the sieve appears nowhere above. The security argument uses only the RSA assumption, the M-LWE assumption, and HKDF's dual-PRF property. Because the sieve's output is statistically identical to any sound generator (Section 5, Appendix B), the standard RSA-2048 assumption applies to the resulting modulus unchanged. There is no "Furui attack," and no special-form attack — Coppersmith with extra structure, Williams's $p+1$, Pollard's $p-1$ — is enabled by the choice of generator.

4.5 Concrete numbers: how much margin is the KDF term?

The reduction adds `2 * Adv_dualPRF(D)`. A fair question from a deploying engineer is: *is that term actually small?* Here is a conservative back-of-the-envelope instantiation; Appendix A.4 gives the reasoning.

The dual-PRF security of HKDF reduces to the PRF security of HMAC-SHA-256, whose best-known bound for an attacker making `q` evaluation queries and running in time `t` is approximately

```
Adv_dualPRF(D)  ~  q^2 / 2^256   +   (PRF break of HMAC-SHA-256)
```

The `q^2 / 2^256` birthday term is the dominant practical contributor. The honest way to weigh it is to compare the bound's two additive terms: the surviving component contributes on the order of `t / 2^192` against an attacker running in time `t`, while the combiner contributes about `q^2 / 2^256` for `q` encapsulations under a single key. Those two terms become equal only near `q ≈ 2^64` — and even there the combiner's value is about `2^128 / 2^256 = 2^-128`, far beyond any feasible attack. For any realistic query budget the combiner term is the *smaller* of the two (at `q = 2^32` it is already down at `2^-192`), and forward-secret rekeying keeps the per-key count far below these limits in any case. The takeaway for an implementer: the combiner is **not** where your security margin goes — for realistic workloads it is set by the surviving component — and the factor of 2 simply accounts for the proof not knowing in advance *which* component that is.

---

5. The Furui sieve: fast RSA prime generation (performance only)

We now describe the prime generator. Keep Section 4.4 in mind throughout: nothing here changes RSA's security; it only changes how quickly primes are found.

5.1 The algorithm

Generating a 2048-bit RSA key needs two ~1024-bit primes. Standard implementations alternate random odd-integer sampling, trial division by small primes, and Miller–Rabin testing, with trial division dominating the cost. The Furui sieve replaces the first two steps over a window of candidates: because every prime greater than 3 has the form 6n±1, it enumerates only those candidates and removes composites, eliminating most of the window before Miller–Rabin runs.

```
FastRSAPrime(target_bits, e):
    while True:
        # pick a random window near the target magnitude
        offset    = random_uniform(1 << (target_bits-1), 1 << target_bits)
        window_lo = offset
        window_hi = offset + (1 << 30)
        primes = FuruiSieve(window_lo // 6, window_hi // 6,
                            segment_size = autotune())
```

```
for p in primes:
    if not (target_bits-1 <= p.bit_length() <= target_bits): continue
    if not MillerRabin(p, k = 64): continue   # k=64 rounds
    if gcd(p - 1, e) != 1: continue
    return p
# no usable prime in this window -> draw a fresh offset
```

5.2 Why the output is identical to a standard generator

The concern with any non-standard prime generator is that it might bias the primes in a way an attacker could exploit. It does not, and the reason is a short rejection-sampling argument.

Both the Furui sieve and a textbook generator (random odd integer → trial division → Miller–Rabin) are *rejection samplers over the same population*: the primes in the chosen range. The sieve's wheel filter and the textbook's trial division are both *conservative* — neither ever discards an actual prime — and at cryptographic sizes Miller–Rabin's false-accept probability is below `2^-128`. Conditioned on a uniformly chosen window, both methods therefore output the uniform distribution over primes in that window. Composing with a uniform choice of window gives the same output distribution for both. The joint distribution of `(p, q)` is the same one any sound generator produces at the same bit length. Appendix B confirms this empirically.

Consequently (restating §4.4 for emphasis): the resulting modulus inherits the ordinary RSA-2048 assumption, and **no special-form attack** — Coppersmith-with-structure, Williams's $p+1$, Pollard's $p-1$ — is opened by the choice of generator.

5.3 Validation, and what to check before production

Appendix B reports two tests on 375,211 sieved primes — a Dirichlet-uniformity test on residue classes and a prime-counting density check — and finds no detectable bias. Those tests run in a small numeric window (around 6–12 million) for the practical reason that the window can be enumerated exhaustively in seconds.

The rejection-sampling argument of §5.2 extends the conclusion to RSA-magnitude primes, but a *belt-and-braces* deployment should not take that on faith. **Before using the sieve to generate production keys, replicate the Dirichlet and density tests at true 1024-bit prime ranges, across several disjoint windows, comparing against the exact reference generator you intend to replace, and publish the raw data and scripts.** The supplied `findprimes.py` already implements the tests; extend it to 1024-bit ranges and record timing and thermal metadata alongside the statistics (Appendix C).

---

6. Implementation guide

This is the section to read before you write code. It is platform-independent; we use Apple Silicon as a concrete worked example because the sieve's data-parallel implementation

maps well to it, but every choice has a direct equivalent on Linux/x86-64 and other Arm platforms.

6.1 Platform and libraries

- **RSA-2048:** use BoringSSL or OpenSSL. **Do not** reach for libsodium here — it provides no RSA by design. On Apple platforms, the FIPS-validated `corecrypto` (via the Security framework) is also an option. Replace the library's prime generation with a wrapper around the Furui sieve (Section 5).
- **ML-KEM-768:** prefer **mlkem-native** [13]. Its C is verified with CBMC and its Arm and x86-64 assembly with HOL Light to be memory-safe and secret-independent (constant-time) at the object-code level — stronger assurance than black-box timing tests can give. PQClean and liboqs are sound alternatives. Build with the target's crypto extensions (e.g. `-march=armv8-a+crypto` on Arm, `-maes` on x86-64), and use a vectorized Keccak for SHA-3/SHAKE, detecting any hardware Keccak (ARMv8.2 SHA-3) extension at runtime rather than assuming it.
- **HKDF:** any audited HMAC-SHA-256 HKDF (CommonCrypto, libsodium, OpenSSL).
- **Build hardening:** follow the OpenSSF Compiler Options Hardening Guide [14] — `-O2`/`-O3`, `-fstack-protector-strong`, `-D_FORTIFY_SOURCE=3`, `-fstack-clash-protection`; where the ABI allows, enable pointer authentication and branch-target identification (BTI) — `-mbranch-protection=standard` on ARMv8.3+, or build for `arm64e` on Apple (not both, as their pointer-authentication schemes are incompatible).
- **Randomness:** draw seeds from the OS CSPRNG (`getentropy(2)`, or `SecRandomCopyBytes` on macOS, `BCryptGenRandom` on Windows), not by opening `/dev/urandom` directly.

6.2 Side-channel mitigations

RSA decapsulation. Use the standard defenses, all present in production libraries: constant-time Montgomery multiplication, ciphertext blinding (`c -> c·r^e` before the private-key operation, then divide `r` out of the result), exponent blinding (`d -> d + k·phi(N)` for fresh `k`), and CRT recombination that is constant-time with respect to whether `p > q`.

ML-KEM decapsulation. The implicit-rejection branch must run in constant time and touch the same memory pattern as the success branch — which mlkem-native's verified build gives you. Confirm the integrated binary with the official constant-time test vectors, **dudect**, and a Valgrind/MemorySanitizer checker such as **TIMECOP** [15]. Run the Valgrind-based checks on a platform Valgrind supports (Linux on x86-64 or aarch64); its macOS/Apple-Silicon support is limited.

Furui sieve and GPU Miller–Rabin (key generation only). Most sieve data is public; the secrets are the random window offset and the chosen prime `p`. Mitigations: draw the offset from a CSPRNG and never branch on its bits; zeroize sieve buffers and candidate lists after use (`explicit_bzero` / `memset_s` / `sodium_memzero`); derive Miller–Rabin witnesses from a fixed sequence (the first 64 primes) so the test is input-independent; keep modular exponentiation constant-time; and when batching on a GPU, fix the batch shape (always `k`

rounds × n candidates) and unroll final selection to a fixed iteration count so wall-clock time never reveals which candidate won.

The supplied reference code (`findprimes.py`, `findprimes_gpu.py`, `primes_mlx.py`; see *Code and data availability*) is for performance demonstration and is **not** constant-time as written; integrate the mitigations above and pass the constant-time checks before generating real keys.

6.3 Production hardening checklist

Treat every item as mandatory before this construction generates or processes real keys.

- ☐ **Constant-time verification** of the integrated ML-KEM *and* RSA decapsulation paths (dudect, TIMECOP/MemorySanitizer), including the failure paths of §3.5.
- ☐ **Implicit rejection** preserved end-to-end: both branches and the combiner always run; no early return reveals which component failed.
- ☐ **Zeroization** of all secret-bearing buffers — RSA blinding factors, the chosen prime, sieve buffers, candidate lists — via `explicit_bzero`/`memset_s`.
- ☐ **Fixed-shape GPU batching** and **deterministic Miller–Rabin witnesses** as in §6.2.
- ☐ **Strict wire parsing:** reject anything whose `key_share` length is not exactly 1440 (ClientHello) or 1344 (ServerHello), and that does not decode as a valid RSA-2048 / ML-KEM-768 ciphertext, *before* calling Decaps (§8.3).
- ☐ **Fuzz every decode path** (the `key_share`/ciphertext parsers) with a coverage-guided fuzzer (libFuzzer, AFL++), ideally continuously via OSS-Fuzz.
- ☐ **Static analysis and sanitizer builds:** clang-tidy / Clang static analyzer / CodeQL in CI; a separate `-fsanitize=address,undefined` (and `=memory`) test build.
- ☐ **Known-answer tests:** validate ML-KEM against the NIST ACVP vectors [16] and the C2SP/CCTV intermediate/negative-path vectors, and the assembled library against Project Wycheproof [17]. If FIPS 140-3 is in scope, pass CAVP/ACVTS.
- ☐ **Supply chain:** pin every dependency by commit hash, generate an SBOM, and verify reproducible builds.

---

7. Performance and sizes

7.1 Wire format

Both components are fixed-length on the wire, so parsing is just splitting at known offsets. We fix the RSA public exponent at `e = 65537` and do not transmit it; only the 2048-bit modulus is sent.

Wire field	Bytes
RSA-2048 modulus `n` (big-endian, fixed length)	256

ML-KEM-768 public key (FIPS 203 §7.1)	1184
Total ClientHello key share	**1440**
RSA-2048 ciphertext (big-endian, fixed length)	256
ML-KEM-768 ciphertext (FIPS 203 §7.2)	1088
Total ServerHello key share	**1344**

7.2 Timings

Single-thread estimates on example hardware (an Apple M2 Pro, representative of a mid-range Arm SoC; microseconds unless noted). Constants differ on other CPUs and accelerators, but the relative costs hold. The full methodology is Appendix C.

Operation	Time
Furui-sieve RSA-2048 KeyGen (this work)	~150 ms
OpenSSL RSA-2048 KeyGen (reference)	~400 ms
ML-KEM-768 KeyGen	~50 µs
Hybrid KeyGen (dominated by RSA)	~150 ms
RSA-2048 Encaps (public-key op)	~100 µs
ML-KEM-768 Encaps	~50 µs
Hybrid Encaps	~160 µs
RSA-2048 Decaps (private-key op, with CRT)	~3 ms
ML-KEM-768 Decaps	~60 µs
Hybrid Decaps	~3.1 ms

7.3 Choosing this versus the alternatives

For a quick operational decision, here is the hybrid next to its two pure components. "Security today" and "after a quantum computer" describe confidentiality of session keys.

Scheme	Handshake latency (Encaps+Decaps)	Public key	Ciphertext	Security today	After a quantum computer
ML-KEM-768 only	~0.1 ms	1184 B	1088 B	~192-bit (lattice; ~10 yr old)	~192-bit
RSA-2048 only	~3.1 ms	256 B	256 B	~112-bit (classical only)	**broken**
Hybrid (this work)	~3.3 ms	1440 B	1344 B	**secure if either holds**	~192-bit (via ML-KEM)

The hybrid's cost is dominated by RSA's private-key operation; ML-KEM is two orders of magnitude faster. You pay RSA's latency and ~256 extra bytes for the classical hedge. When that hedge stops being worth it (Section 9), the natural successor is pure ML-KEM-768 — or ML-KEM-1024, optionally with an X25519 hedge — at a fraction of the latency.

---

8. Operations

8.1 Key-rotation policy

RSA-2048 is acceptable as the classical half **only for the transition window** — roughly until a quantum computer can factor 2048-bit moduli. Calibrated to the Gidney–Ekerå estimates [3] and the CNSA 2.0 timeline [4]:

- **Ephemeral RSA keys** (per-handshake in TLS/HPKE): generated fresh per session, no rotation needed beyond the session. The Furui sieve makes this cheap.
- **Long-term RSA-2048 identity keys:** rotate at most annually, preferably semi-annually, until 2030. After 2030, treat any RSA-2048 key whose ciphertexts may have been recorded as compromised, and migrate the identity to ML-DSA (FIPS 204 [18]) or SLH-DSA (FIPS 205 [19]).
- **Retirement target:** 2032–2035 for confidentiality (consistent with CNSA 2.0), earlier if CRQC estimates accelerate.
- **Long-lived signed artifacts** (firmware, code-signing, archives): do not use RSA-PSS for signatures meant to be valid past 2030; use SLH-DSA or ML-DSA, optionally hybridized with Ed25519 for the early period.

8.2 Operational playbook

A compact, one-page plan for running this through the migration:

1. **Key-generation policy.** Default to **ephemeral** hybrid keys for transport. Reserve long-term RSA identity keys for cases that genuinely need them, and tag each with a hard expiry date enforced in code.
2. **Rotation automation.** Wire rotation to your secrets manager: generate-and-deploy on a fixed schedule (§8.1), alert on keys approaching expiry, and make "rotate now" a single operation you can trigger on news of a cryptanalytic result.
3. **Ciphertext logging and retention.** Because harvest-now-decrypt-later is the core threat, **minimize how long recorded ciphertexts remain decryptable**: set explicit retention limits on logged/transcripted traffic, and record, per data class, how long its confidentiality must hold — that horizon is what decides whether RSA's eventual fall matters for it.
4. **Migration triggers.** Tie concrete actions to public milestones: track the NIST PQC migration timeline, CNSA 2.0 [4], and BSI [5] advisories, and credible CRQC progress; pre-define the thresholds at which you (a) shorten RSA rotation, (b) stop issuing new

RSA identity keys, and (c) drop the RSA component entirely and move to pure ML-KEM.

8.3 Downgrade resistance and parser checks

A network attacker cannot strip the post-quantum component without breaking the transcript binding in `info` (§3.4) — *provided* your parser is strict. Enforce exact lengths (1440 for ClientHello, 1344 for ServerHello) and validate that each component decodes correctly **before** calling Decaps; abort the handshake on any mismatch rather than silently truncating. This length check is the primary defense against negotiation-stripping and downgrade-to-classical attacks. For TLS 1.3 the construction maps onto a combined named group in the hybrid key-exchange design [10]; for HPKE [20] it fits as a custom KEM ID in the experimental range `0xFE00–0xFEFF`.

---

9. Limitations and honest scope

Hybrid is a transition, not a destination. Once quantum computers mature, RSA's contribution drops to zero security and becomes pure overhead in latency, bandwidth, and key management. This design is engineered for the pre- and early-CRQC window; the steady-state successor is pure ML-KEM-768 (or -1024), possibly with an X25519 hedge during the early-CRQC period.

ML-KEM is comparatively young. Module-LWE has been studied since around 2010 and ML-KEM since 2017 — short next to RSA's four decades. The hybrid exists precisely to hedge that youth; once ML-KEM has accumulated comparable scrutiny, the hybrid becomes pure cost.

The sieve is orthogonal. It is a drop-in key-generation optimization with no security role (Sections 4.4, 5.2). We include it because it is genuinely useful on accelerator hardware and because it concretely answers "where can a 6n±1 sieve fit in post-quantum cryptography?" — namely, in classical key generation, never in the security-providing component.

Long-term key exposure. Hybrid mode protects forward-secret *session* keys, not a long-term RSA private key, which a future quantum computer could recover retroactively. See §8.1.

This design has not had independent expert review. As stated on the front page, the formal reduction (Appendix A) and the constant-time implicit-rejection handling (§3.5, §6.2) in particular should be audited by a cryptographer with lattice and protocol expertise before deployment. The numbers in §4.5 are conservative engineering estimates, not a substitute for that review.

---

10. Conclusion

We have described a hybrid KEM that combines RSA-2048 with ML-KEM-768 under a dual-PRF HKDF combiner, inheriting IND-CCA security from whichever component is still standing and degrading gracefully to pure ML-KEM once RSA falls. The Furui sieve rides along as a security-neutral speed-up for classical key generation, with both a proof and empirical evidence that its output is indistinguishable from a standard generator. The construction, the implementation checklist, and the operational playbook are meant to be a usable reference for transition-era deployments — pending the independent cryptographic review this paper repeatedly, and deliberately, asks for.

Appendix A. The formal IND-CCA reduction

A.1 Statement

Let RSA-KEM and ML-KEM-768 be IND-CCA-secure KEMs with 32-byte shared secrets, and let HKDF be the extract-then-expand construction instantiated with HMAC-SHA-256. For any `(t, q)`-bounded IND-CCA adversary `A` against the hybrid (`t` total time, `q` decapsulation queries),

```
Adv_hybrid(A)  <=  min{ Adv_RSA(B_R),  Adv_MLKEM(B_M) }  +  2 · Adv_dualPRF(D)
```

where `B_R`, `B_M`, and `D` are explicit reductions each running in time `t + O(q · (T_RSA + T_MLKEM + T_HKDF))` and making at most `q` oracle queries, and `T_X` is the per-call cost of primitive `X`.

A.2 Game hops

Define games `G0` … `G3`, where `G0` is the IND-CCA experiment for the hybrid and `G3` replaces the challenge shared secret with fresh randomness independent of the challenge ciphertext.

- **G0 → G1.** Replace `ss_RSA` in the challenge with a uniformly random value. Any distinguisher is an IND-CCA adversary `B_R` against RSA-KEM (it simulates the ML-KEM and HKDF parts itself and forwards RSA decapsulation queries to its own oracle), so the gap is at most `Adv_RSA(B_R)`.
- **G1 → G2.** The challenge output is now `HKDF(random || ss_MLKEM, info)`. By HKDF's dual-PRF property (keyed by the now-random first input), replace it with output keyed by independent randomness; the gap is at most `Adv_dualPRF(D_1)`.
- **Symmetric path.** The alternative hops that replace `ss_MLKEM` first cost `Adv_MLKEM(B_M)` then `Adv_dualPRF(D_2)`. The adversary must beat *one* of the two paths, so the bound takes the **minimum** of the two component advantages and adds the dual-PRF term **twice** (once per path) — the source of the factor 2.

- **In G3** the challenge secret is independent randomness, so the adversary's advantage is exactly 0. Composing the gaps yields the theorem.

A.3 Simulating decapsulation queries

Each reduction must answer `A`'s `q` decapsulation queries without the secret key it is attacking. `B_R` knows the ML-KEM secret key (it generated it) and the HKDF logic, and forwards the RSA part of each query to its own RSA-KEM oracle; `B_M` does the mirror image. The implicit-rejection behavior of both components (§3.5) is what makes this work: a malformed query yields a deterministic pseudorandom value the simulator can compute on its own, so no decapsulation query leaks a secret. `D`'s simulation just evaluates HKDF in the appropriate keying direction.

A.4 Tightness and a concrete instantiation

The reduction is tight up to the factor of 2 on the dual-PRF term, which is unavoidable for this combiner template (the adversary may exploit either keying direction). The per-query overhead `T_RSA + T_MLKEM + T_HKDF` is constant, so for realistic deployments where `q` is small relative to `t`, the reductions run in essentially time `t`.

To make §4.5 concrete: HKDF's dual-PRF advantage reduces to the PRF security of HMAC-SHA-256. Modeling the compression function as a PRF, the standard bound for an adversary making `q` queries is dominated by the birthday term `q^2 / 2^256` plus the underlying PRF-break term (itself negligible for SHA-256 under standard assumptions). At a deliberately extravagant `q = 2^64`,

```
2 · Adv_dualPRF(D)  ~  2 · ( 2^128 / 2^256 )  =  2^-127
```

an absolute advantage far beyond any feasible attack. The meaningful comparison is between the bound's two additive terms: the component term scales as `~t / 2^192` and the combiner's birthday term as `~q^2 / 2^256`, so the two cross only near `q ≈ 2^64` queries against a single key. Below that — i.e. for every realistic workload, and certainly within the per-key limits forward-secret rekeying enforces — the combiner term is the smaller of the two, and the hybrid's security is set by the surviving component, not the KDF. (For component-level concrete bounds at these parameters, consult the FIPS 203 [6] and KEM-combiner [9] analyses.)

Appendix B. Empirical validation of the sieve's output

We test the §5.2 claim that the sieve's output is unbiased. Both tests are reproducible from the supplied `findprimes.py`.

B.1 Setup

We sieve every prime in the window `W = [6,000,005, 12,000,001]` (the call `find_primes(x=1,000,001, y=2,000,000)`), obtaining the exact set of all 375,211 primes greater than 3 in `W`. We do not sub-sample; the test population is the complete prime set in the window. The window is small enough to enumerate exhaustively in seconds yet large enough for stable statistics; the rejection-sampling argument (§5.2) carries the conclusion up to RSA-magnitude primes.

B.2 Test 1 — Dirichlet uniformity of `p mod q`

By Dirichlet's theorem, primes are equidistributed across the residue classes coprime to a modulus `q`. We test this with a Pearson chi-square goodness-of-fit against the uniform distribution for small primes `q`.

q	χ^2	dof	p-value	Result ($\alpha = 0.05$)
5	0.212	3	0.9701	pass
7	0.478	5	0.9910	pass
11	0.496	9	0.9999	pass
13	1.303	11	0.9997	pass
17	1.201	15	1.0000	pass
19	2.243	17	1.0000	pass
23	2.177	21	1.0000	pass
29	3.358	27	1.0000	pass
31	3.657	29	1.0000	pass
37	8.173	35	1.0000	pass
41	4.780	39	1.0000	pass
43	7.255	41	1.0000	pass
47	7.307	45	1.0000	pass
53	6.556	51	1.0000	pass
97	19.775	95	1.0000	pass

Every statistic is far below the $\alpha = 0.05$ critical value. The very high p-values reflect that we test the *complete* population in the window, not a sample. No bias against uniformity is detected.

B.3 Test 2 — Prime-counting density

The prime number theorem predicts the count of primes in `[a, b]` as roughly `b/ln b − a/ln a`.

Quantity	Value
PNT estimate `b/ln b − a/ln a`	351,741

Sieve-observed count	375,211
Ratio observed / PNT	1.067
Result	pass (ratio ≈ 1 ± O(1/ln a) for windows of this size)

The 6.7% over-count is the expected residual of the `x/ln x` approximation; the more precise `Li(x)` estimate agrees within a fraction of a percent. The sieve enumerates the full prime population with no omissions.

B.4 Conclusion and the 1024-bit replication plan

Both tests are consistent with §5.2: the sieve produces the complete, unbiased set of primes in its window. Before production use, **replicate at 1024-bit prime sizes** (the size RSA-2048 actually draws), across **multiple disjoint windows**, comparing against the exact reference generator the deployment will replace, and publish the raw outputs, the scripts, and timing/thermal metadata. Exhaustive enumeration is infeasible at that scale, so the 1024-bit study should sample large fixed counts per window and compare the same Dirichlet and density statistics between the sieve and the reference generator.

Appendix C. Reproducible benchmark methodology

The Section 7 figures are calibrated engineering estimates. This appendix specifies how to reproduce or supersede them on your own hardware.

C.1 Hardware identification

Report, using the platform's equivalents: CPU model and core counts, total RAM, accelerator/GPU model and core count, OS and kernel version, and thermal state at the start and end of the run. On the example macOS/Apple Silicon target these come from `sysctl machdep.cpu.brand_string`, `sysctl hw.memsize`, `system_profiler SPDisplaysDataType`, `sw_vers`, and `pmset -g thermlog`; on Linux use `/proc/cpuinfo`, `/proc/meminfo`, `lspci/nvidia-smi`, `uname -a`, and the platform thermal interface.

C.2 Build configuration

Record the compiler and exact flags. Use `-O3` plus the target's crypto extensions for production, and add the hardening flags of §6.1 for defensive builds, following the OpenSSF guide [14]. Maintain a separate instrumented test build with `-fsanitize=address,undefined` (and `=memory` for the constant-time checks), and run clang-tidy, the Clang static analyzer, and CodeQL in CI, plus coverage-guided fuzzing of the parsers. Pin ML-KEM (mlkem-native [13], PQClean, or liboqs) and RSA (BoringSSL/OpenSSL) by commit hash.

C.3 Randomness

Draw all keygen seeds from the OS CSPRNG (`getentropy`, `SecRandomCopyBytes`, or `BCryptGenRandom`), stretched via AES-CTR-DRBG (NIST SP 800-90A) with ≥256 bits of entropy. Include DRBG time in the keygen measurement. Do not pre-seed with deterministic vectors except for interoperability cross-checks, reported separately.

C.4 Measurement protocol

- **Warmup.** Discard the first 1,000 operations of each kind to let JIT compilation, accelerator kernel caching (e.g. MLX on the example target), and caches settle.
- **Sample size.** At least 10,000 operations per measurement, system otherwise idle.
- **Statistics.** Report median, 5th and 95th percentiles, and standard deviation; report single- and multi-thread runs separately.
- **Thermal exclusion.** Monitor the platform's thermal state throughout (`pmset -g thermlog` on the example macOS target) and discard any measurement after a thermal-throttle event until the device returns to nominal for ≥60 s.
- **Allocation included.** Time buffer allocation and finalization too, since they are unavoidable in production.

C.5 Reporting

Publish raw timing logs (CSV: operation, run index, elapsed ns, thermal state) alongside summary statistics, plus the exact source or upstream commit hashes for each component.

C.6 Test vectors

For interoperability, publish: (1) at least 16 complete `(pk, sk, ct, ss)` tuples with documented seeds, covering both success and rejection paths; (2) the salt and version tag used in HKDF; (3) the exact byte order of `ct_R || ct_M` and `ss_RSA || ss_MLKEM`. Verify byte-exact agreement of derived secrets across independent implementations. Beyond these construction-specific vectors, validate the ML-KEM component against the NIST ACVP vectors [16] and the C2SP/CCTV intermediate and negative-path vectors, and the assembled library against Project Wycheproof [17]; if FIPS 140-3 validation is in scope, pass CAVP/ACVTS.

Code and data availability

The reference implementations — `findprimes.py` (CPU / NumPy), `findprimes_gpu.py` (CUDA / CuPy), and `primes_mlx.py` (Apple Silicon / MLX) — together with the prime set behind Appendix B, are in the project repository at https://github.com/r-coin/basic/tree/master/security%20tools.
As noted in §6.2, this code demonstrates the sieve's performance and is **not** constant-time as written; apply the hardening of §6.3 before using it to generate keys.

References

[1] R. Furui, “The formulation of prime numbers,” 2024. http://github.com/r-coin/basic/blob/master/security%20tools/Prime%20numbers.pdf.

[2] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994, pp. 124–134.

[3] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” Quantum, vol. 5, p. 433, April 2021.

[4] National Security Agency, “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0),” September 2022.

[5] Bundesamt für Sicherheit in der Informationstechnik (BSI), “Technical Guideline TR-02102-1: Cryptographic Mechanisms — Recommendations and Key Lengths,” 2024.

[6] National Institute of Standards and Technology, FIPS 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” August 2024.

[7] J. Randall, B. Kaliski, J. Brainard, S. Turner, “Use of the RSA-KEM Key Transport Algorithm in the Cryptographic Message Syntax (CMS),” RFC 5990, September 2010.

[8] N. Bindel, U. Herath, M. McKague, and D. Stebila, “Transitioning to a quantum-resistant public key infrastructure,” Post-Quantum Cryptography (PQCrypto 2017), Springer LNCS 10346.

[9] F. Giacon, F. Heuer, and B. Poettering, “KEM combiners,” Public-Key Cryptography (PKC 2018), Springer LNCS 10769.

[10] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid key exchange in TLS 1.3,” IETF Internet-Draft draft-ietf-tls-hybrid-design.

[11] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010.

[12] M. Bellare and B. Tackmann, “The multi-user security of authenticated encryption: AES-GCM in TLS 1.3,” CRYPTO 2016, Springer LNCS 9814.

[13] Post-Quantum Cryptography Alliance (Linux Foundation), PQ Code Package, “mlkem-native: a fast, portable, and formally verified (CBMC + HOL Light) C90 implementation of ML-KEM / FIPS 203,” v1, 2025. github.com/pq-code-package/mlkem-native

[14] Open Source Security Foundation (OpenSSF) Best Practices Working Group, “Compiler Options Hardening Guide for C and C++,” 2024. best.openssf.org.

[15] D. J. Bernstein et al., “TIMECOP: automated constant-time verification in SUPERCOP,” post-apocalyptic-crypto.org/timecop; supersedes A. Langley, “ctgrind” (2010). See also the constant-time tool survey, CRoCS, crocs-muni.github.io/ct-tools.

[16] National Institute of Standards and Technology, ACVP ML-KEM test vectors, Automated Cryptographic Validation Program, github.com/usnistgov/ACVP-Server; intermediate and negative-path vectors at github.com/C2SP/CCTV.

[17] C2SP, “Project Wycheproof: test vectors that probe cryptographic libraries against known attacks, including ML-KEM,” github.com/C2SP/wycheproof.

[18] National Institute of Standards and Technology, FIPS 204, “Module-Lattice-Based Digital Signature Standard,” August 2024.

[19] National Institute of Standards and Technology, FIPS 205, “Stateless Hash-Based Digital Signature Standard,” August 2024.

[20] R. Barnes, K. Bhargavan, B. Lipp, C. Wood, “Hybrid Public Key Encryption,” RFC 9180, February 2022.